~~83%~~

90%

DATABASE MANAGEMENT SYSTEMS

ICT

MODULE 4

CAT 3

Candidate Name: KAIRU MARCUS

Admission Number: 578 **Date:** 28/10/2026

Time: 1 hour | **Total:** 30 marks

Instructions

1. Answer ALL questions.
2. Write your name and admission number in the spaces provided above.
3. Show all working and write your answers neatly.

QUESTIONS

1. Define **normalization** and state its main purpose. (2 marks)
2. Briefly explain the three types of anomalies caused by poorly designed tables. (2 marks)
3. Differentiate between **CHAR** and **VARCHAR**, and state which data type you would use for storing prices and why. (2 marks)
4. State **two advantages** and **two disadvantages** of indexing. (2 marks)

5. Distinguish between a **PRIMARY KEY** and a **UNIQUE** constraint. (2 marks)
6. A sports club keeps the following table:

CLUB_MEMBER

MemberID	MemberName	Sports
1	Mary	Football, Netball
2	John	Chess
3	Peter	Football, Chess, Swimming

a) Explain why this table is not in 1NF. (1 mark)

b) Convert the table into 1NF and state its primary key. (3 marks)

7. Consider this table, which is in 1NF, with composite primary key {OrderID, ProductID}:

ORDER_ITEM

OrderID	ProductID	ProductName	UnitPrice	Quantity
O1	P1	Laptop	60000	2
O1	P2	Mouse	800	5
O2	P1	Laptop	60000	1

a) List the functional dependencies in the table and identify any partial dependency. (2 marks)

b) Decompose the table into 2NF. (2 marks)

c) Give **one** anomaly that the original table suffers from. (1 mark)

8. a) Write an SQL CREATE TABLE statement for **ENROLLMENT** with these attributes:

   - StudentID (integer)
   - CourseID (up to 6 characters)
   - EnrollDate (date, defaulting to the current date)
   - Marks (must be between 0 and 100)

   The primary key is {StudentID, CourseID}. StudentID references STUDENT(StudentID) and CourseID references COURSE(CourseID). (3 marks)

   b) Write a CREATE INDEX statement to index the StudentName column of the STUDENT table, and give one reason why this column is a good candidate for indexing. (2 marks)

9. Use the tables below to answer this question.

   STUDENT(StudentID, StudentName, Department)
   ENROLLMENT(StudentID, CourseID, EnrollDate, Marks)

   a) Write a SELECT query to display the names and departments of all students in the 'ICT' department, arranged alphabetically by name. (2 marks)

   b) Write a query to display the average marks for each CourseID, showing

only courses whose average is above 60. (2 marks)

c) Write a query that joins the two tables to display each student's name, CourseID and Marks. (2 marks)

Database.

1. Normalization is process of organizing data in relational database to minimize redundancy

Main Purpose: To eliminate data anomalies and ensure data integrity

2. Insertion anomaly - Inability to insert data into a table without providing values for other unrelated attributes.

Update anomaly: When data is stored multiple times and only some of that is updated

Deletion anomaly: Unintended loss of data when a record is deleted.

3. CHAR. It always uses the full declared length.

VARCHAR. It stores only the actual characters used.

4. Indexing

Advantages

It speeds up data retrieval

Enforces uniqueness when a unique index is used.

Disadvantages

- Slows down INSERT, UPDATE & DELETE operations coz the index must also be maintained
- Consumes extra disk space.

6. Primary key uniquely identifies each row and does not allow Null values with only one Primary key per table.

A unique constraints also enforces uniqueness but allows one Null value and can have many unique constraints in a table.

6. The sports contains ~~multivalued data~~ and values repeating. It's not atomic

Member ID	Member name	Sport
1	Mary	Football
1	Mary	Netball
2	John	Chess
3	Peter	Football
3	Peter	Chess
3	Peter	Swimming

9 + 1 = 10

7. a) Functional dependency : OrderID, ProductID → Quantity
Partial dependency → ProductID → ProductName and UnitPrice

b) Products

ProductID	ProductName	UnitPrice
P1	Laptop	60000
P2	Mouse	500

OrderItems

OrderID	ProductID	Quantity
O1	P1	2
O1	P2	5
O2	P1	1

c) Update anomaly

8. CREATE TABLE ENROLLMENT (
StudentID INT,
CourseID VARCHAR(6),
Enrolldate DATE DEFAULT current_date,
Marks INT CHECK (marks BETWEEN 0 AND 100),
PRIMARY KEY (StudentID, CourseID),
FOREIGN KEY (StudentID) REFERENCES STUDENT (StudentID),
FOREIGN KEY (CourseID) REFERENCES COURSE (CourseID)
);

b. CREATE INDEX idx_Studentname ON STUDENT (Student_name);
Student_name is frequently used in search conditions and ORDER BY clauses.

c. a) SELECT StudentName, Department
FROM STUDENT
WHERE Department = 'CT'
ORDER BY StudentName; ASC | DESC

b) SELECT CourseID, AVG(Marks) AS AverageMarks
FROM ENROLLMENT
GROUP BY CourseID
HAVING AVG(Marks) > 60

c. SELECT S.StudentName, E.CourseID, E.Marks
FROM STUDENTS S
JOIN ENROLLMENT E ON S.StudentID = E.StudentID;